

黑匣子 Cloudflare 网站技术归档

DESIGN REPORT · 2026 / 09 / 26

阅读范围与结论

阅读基准：2026 年 9 月 26 日的仓库代码与配置。本文解释这套网站如何工作，也区分“代码中存在”“历史记录曾验证”和“当前线上尚未核验”。面向第一次接触项目的维护者。可打印版本见同目录的 DOCX 与 PDF。

系统构成

单 Worker · D1 元数据 · R2 文件

01 / SECTION

一 项目背景与目标

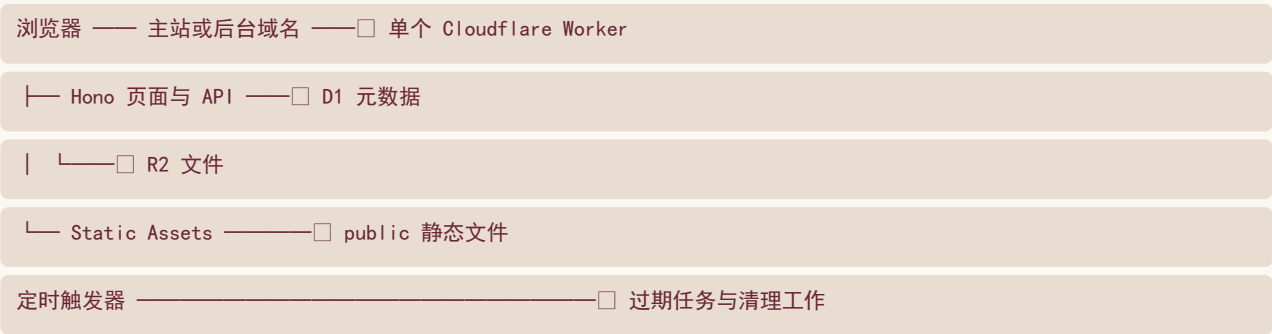
黑匣子网站保存校园戏剧社团的作品、演职人员与相关资料。读者可能只想查一部剧的演出版本和参与者；成员可能要补充剧本、照片或音频；管理员则需要核对资料、管理可见范围。网站把这三种需求放进同一套资料模型：作品是被归档的剧目，版本区分同一剧目的不同演出，成员与演职记录把人和版本联系起来，资源则指照片、剧本、音频等文件及其描述。对应关系可从 数据库迁移 [migrations/0001_core.sql]、版本迁移 [migrations/0015_production_editions.sql] 及作品页面 [src/routes/productions.ts] 核对。

系统的目标是让公开资料可浏览、让投稿有审核入口、让后台操作留下可追踪的数据。它目前是一个由 Cloudflare Worker 承载的网站，并非几个独立部署的前后端服务。原始技术选型时为何采用 Cloudflare，仓库没有足够的决策记录；本文只解释现有实现及其影响，不倒推当时的动机。

02 / SECTION

二 系统整体架构

浏览器访问主站 npublackbox.online 或后台域名 admin.npublackbox.online 时，请求进入同一个 Worker。Worker 使用 Hono 路由，动态页面在服务端生成 HTML；静态脚本、样式和图片由 Static Assets 绑定提供。D1 保存成员、作品、资源元数据及审核状态，R2 保存上传文件的字节。这里的 D1 是 Cloudflare 的 SQL 数据库绑定，R2 是对象存储；前者回答“这是什么、谁能看”，后者保存“文件内容是什么”。绑定与域名见 wrangler.jsonc [wrangler.jsonc]，入口和路由装配见 src/index.ts [src/index.ts]。



两个域名承担不同的入口职责，但后台没有单独服务器。后台域名的访问门禁由 admin-portal 中间件 [src/middleware/admin-portal.ts] 执行。Worker 每小时第 17 分钟收到一次定时触发，处理过期上传、限流记录和导入明细清理；这是一项后台维护工作，并不替代人工监测与备份。配置 [wrangler.jsonc] 和定时入口 [src/index.ts] 可核对触发时间及任务。

03 / SECTION

三 核心技术选择及其影响

读者需要先知道请求在哪里被处理。页面与 API 共用一个 Hono 应用：中间件先建立请求上下文、加安全响应头、校验同源写入、加载用户，再进入具体路由。入口 `[src/index.ts]` 展示了这个顺序。因此路由可以复用同一份身份和数据库绑定；改动公共中间件也会同时影响主站与后台，需要回归两类页面。

服务端渲染使首页、作品页与资源页可直接返回完整 HTML。首页 `[src/routes/home.ts]` 并行读取站点资料、公告、焦点内容和最新作品，再交给页面模板。静态资源走 `Assets` 绑定；未命中动态路由的请求才交给 `Assets`。入口 `[src/index.ts]` 及配置 `[wrangler.jsonc]` 说明了这个边界。这样部署对象只有一个 Worker，但静态文件与动态权限逻辑仍分开。

数据访问使用 D1 原生绑定的 `prepare`、`bind`、`first`、`all`、`run` 和 `batch`，并未建立传统数据库连接池或使用 Hyperdrive。绑定类型 `[src/types.ts]` 和实际查询可直接核对这一点。历史评估见 `git show 2bbe8a4:docs/数据库连接管理评估.md`；这里以当前代码为准。R2

用于存储文件，不作为资源目录：文件是否可见先由 D1 判断。这个分工让权限规则集中在应用层，也意味着 D1 与 R2 无法共用一次原子事务，后文会说明补偿清理。

04 / SECTION

四 关键请求与数据流

公开浏览如何到达作品和资源

访客从首页进入作品列表，可搜索并查看作品详情；作品详情把版本、演职记录和已批准资源组织在一起。作品路由 `[src/routes/productions.ts]` 处理列表、时间线及详情。隐藏作品只对管理员可见，关联资源也随之受限，可见性判断 `[src/services/production-visibility.ts]` 是公共入口，避免只隐藏页面而仍能取得文件。

资源有两层访问：访客可看已批准资源的列表或预览；完整详情及文件下载要求登录。下载时先读 D1 中的资源及关联作品状态，再从 R2 取文件，设置安全的文件类型、下载头、范围请求与缓存头。资源路由 `[src/routes/resources.ts]` 和文件服务 `[src/services/resource-files.ts]` 共同实现这个顺序。即使知道 R2 对象键，也不能跳过网站的资源权限检查。

GET `/productions/:id` → 查作品可见性 → 读版本和演职记录 → 读已批准资源 → 返回 HTML

GET `/resources/:id/...` → 查资源与作品状态 → 检查登录和权限 → 读 R2 → 返回文件

一次投稿从上传到公开

投稿者先登录并提交文件信息。上传路由检查 CSRF 令牌（防止其他网站借用已登录浏览器发起写入）、文件类别、大小、作品与版本关系及同时进行的任务数，然后创建最长 24 小时的上传任务与空间预留。上传路由〔src/routes/uploads.ts〕、上传策略〔src/services/upload-policy.ts〕可核对这些规则。常规路由由 Worker 分片接收、写入 R2；若运行环境同时配置 R2 账号、访问密钥及桶名，代码也能生成直传所需的信息。仓库的生产配置没有声明这组直传凭据，线上 Secret 状态尚未核验，因此不能断言当前线上已启用直传。

完成上传后，系统合并对象并写入资源记录。管理员或符合代码中低风险规则的已登录成员投稿可自动批准，其余进入待审核状态；“文件已进入 R2”不等于“访客已经能看”。完成流程〔src/services/uploads.ts〕和资源查询〔src/routes/resources.ts〕给出实际状态转换。失败或撤销留下的对象由清理队列处理，见第五节。

后台批量导入如何避免半成品数据

后台可先预览演职人员表格，再确认导入；输入有列数、行数和文件大小限制。确认和受保护的回滚使用 D1 批处理，减少只写入一部分演职关系的风险。导入路由〔src/routes/credit-import.ts〕、格式约束〔src/services/credit-import/schema.ts〕、提交逻辑〔src/services/credit-import/commit.ts〕可核对。旧材料中“导入只在本地”的说法与后续发布记录相冲突：仓库历史记录显示 V3.5 已部署相关功能，但本次没有登录线上后台重新执行导入。

05 / SECTION

五 数据库 文件与媒体存储

D1 存储账号、成员、作品、版本、演职记录、资源、上传任务、审核及导入记录。数据库结构通过编号迁移演进，目前仓库有 19 个迁移文件，最新是 0019 演职导入〔migrations/0019_credit_imports.sql〕。维护者应先检查迁移状态，再备份并执行生产迁移，不能把本地数据库已更新当作生产已更新。2026 年 9 月 22 日的历史发布记录称生产已应用至 0019；当前线上迁移状态待重新核验。

R2 保存照片、剧本、音频等原始字节。代码明确关闭视频上传；照片最多 20 MiB、剧本 50 MiB，音频及其他文件 100 MiB，单片 50 MiB。9 GB 是应用层预算，综合已有对象和预留空间计算，并非 Cloudflare 账户或桶的真实容量上限。上传策略〔src/services/upload-policy.ts〕给出精确值。资源表仍是文件检索与权限的入口；不要直接把桶内容当作公开站点目录。

D1 的批处理只能保证 D1 内相关写入的原子性，不能把 R2 对象写入一并纳入。实现用“先写对象、再确认记录；失败时记清理意图并重试”的补偿办法处理跨存储不一致。上传完成〔src/services/uploads.ts〕、文件清理〔src/services/file-cleanup.ts〕和定时任务〔src/index.ts〕形成闭环。此设计减少孤儿文件，但依赖清理任务持续运行；它不是跨 D1、R2 的强事务。

图片字节可在边缘 Cache API (Worker 可读写的缓存接口) 中短期复用。每次请求仍先做 D1 权限判断与 R2 对象检查, 再考虑读取内部缓存; 给浏览器的响应仍使用私有缓存约束。文件服务 [src/services/resource-files.ts] 及边缘缓存 [src/services/edge-cache.ts] 说明了缓存边界。站点设置另有短时间应用缓存, 站点资料服务 [src/services/site-profile.ts] 在修改时失效, 但不应据此承诺所有边缘位置瞬时一致。

生产备份脚本导出 D1 SQL 到本地的忽略目录; 它不包含 R2 文件。部署脚本 [scripts/predeploy.mjs] 中 backup 分支可核对。若需可恢复的完整归档, 还必须另外定义 R2 备份、版本和恢复演练, 仓库未提供经验证的端到端方案。

06 / SECTION

六 权限与安全

用户登录后获得 HMAC-SHA256 签名的会话令牌; 这种签名让服务端能检查令牌是否被篡改。浏览器保存令牌的 Cookie (随请求发送的小型凭据) 设置为 HttpOnly、SameSite=Lax, HTTPS 下带 Secure, 有效期 14 天。每次需要身份的请求仍按令牌中的用户标识与认证版本查询 D1, 以检查账号状态和强制改密条件。令牌 [src/auth/session.ts]、Cookie [src/http/cookies.ts]、用户加载 [src/middleware/session.ts] 分别承担这些职责。密码用 scrypt 保存, 同时兼容历史格式的验证, 密码实现 [src/auth/password.ts] 可核对。

权限不是只看是否登录。路由区分访客、普通成员与管理; 隐藏作品只让管理员访问, 关联资源同样受限; 上传、下载、审核和批量导入又各有入口检查。作品可见性 [src/services/production-visibility.ts]、资源路由 [src/routes/resources.ts]、后台门禁 [src/middleware/admin-portal.ts] 是审查权限时应连读的文件。后台域名只是同一 Worker 的独立入口, 不提供天然的第二层服务器隔离。

写入请求先受 Origin 与 Sec-Fetch-Site 同源检查, 再由需要写入的路由检查 CSRF 令牌。页面还设置内容安全策略、禁止嗅探与框架嵌入等响应头; 敏感页面和错误响应使用 no-store。安全中间件 [src/middleware/security.ts] 实现这些规则。生产配置开启 Turnstile, 但代码把挑战用于注册和匿名反馈等指定流程, 而不是每个请求; 服务端会校验挑战动作和主机名, Turnstile 服务 [src/services/turnstile.ts] 可核对。限流记录放在 D1, 限流中间件 [src/middleware/request-limits.ts] 负责读取与更新。

07 / SECTION

七 部署和运维

本地需要 Node.js 24 或更新版本。安装依赖后运行 npm run db:init 建立本地 D1, 再用 npm run dev 启动开发 Worker; 命令与版本约束见 package.json [package.json]。本地配置使用名为 blackbox-local 的 D1 与 blackbox-files 的 R2 绑定, 不应把本地成功视作已验证生产环境。

生产发布是显式操作，而不是 `git push` 自动触发。当前脚本的顺序应理解为：`npm run deploy:check` 执行格式、类型、后端测试和生产配置 `dry run`（只模拟构建，不发布）；`npm run db:production:backup` 导出 D1；`npm run db:production:migrate` 应用远端迁移；`npm run deploy` 发布 Worker；`npm run smoke:production` 对九个公开端点做基本响应检查。命令定义〔`package.json`〕、部署脚本〔`scripts/predeploy.mjs`〕与生产绑定〔`wrangler.jsonc`〕共同说明这条路径。`deploy:check` 本身不会发布，冒烟检查也无法代替登录、上传与后台操作验收。

生产 Worker 在两个自定义域名上运行，D1 与 R2 通过 DB、FILES 绑定访问；生产环境声明会话与 Turnstile Secret 为必需项，具体值不应进入仓库。`wrangler.jsonc`〔`wrangler.jsonc`〕仅能证明声明，不能证明 Cloudflare 控制台中资源和 Secret 当前状态。配置启用 Workers 可观测性；错误中间件〔`src/middleware/errors.ts`〕只记录请求 ID、路由、状态码和总耗时等有限信息，没有逐条 SQL 查询耗时。故不能仅凭代码给出线上延迟或数据库瓶颈结论。

仓库历史中的最近一份生产验证记录对应 2026 年 9 月 23 日的成员入社年份筛选发布；更早的 V3.5 记录在 9 月 22 日列出 0019 迁移、自动测试与九项冒烟检查。它们是当时的发布证据，不证明 9 月 26 日线上仍是同一版本，也不证明真实用户流程全部成功。历史记录已从工作树归档删除，可用 `git show 2bbe8a4:docs/验证记录.md` 与 `git show 2bbe8a4:docs/releases/V3.5.md` 查阅。

08 / SECTION

八 已知限制与后续方向

当前代码不开放视频上传。直传 R2 的实现存在，但当前线上凭据及启用状态待核验。D1 SQL 备份不覆盖 R2 文件，完整恢复路径仍需设计与演练。历史发布记录有测试和公开端点冒烟检查，但没有本次的线上权限、上传、后台导入或恢复演练结果。现有日志也不足以量化单条 D1 查询或端到端媒体访问表现。这些应作为下一轮运维核验和观测任务，而不是写成已完成能力。

仍待确认的历史问题有两项：最初选择 Cloudflare 组合的决策过程，以及部分审核规则当时的业务依据。代码能证明系统现在怎样工作，不能证明当初为什么决定这样做。若后续找到会议纪要或负责人确认，可把依据补入本文，并保留核验日期。

09 / SECTION

写法参考与核验入口

本文通过 GitHub 工具查看了持续维护、星标较多且与本项目技术表达有关的项目，只借鉴组织方式：

项目 · 参考点 · 本文的使用方式

Hono README [https://github.com/honojs/hono/blob/main/README.md] ·

先用一句话建立框架定位，再给最小请求示例和深入入口 · 先解释单 Worker，再用两条请求路径展示运行方式

Directus README [https://github.com/directus/directus/blob/main/readme.md] ·

从读者能完成的任务组织能力，再链接到细节 · 以浏览、投稿、后台导入串起路由与存储

Cloudflare Workers SDK README [https://github.com/cloudflare/workers-sdk/blob/main/README.md] ·

快速开始与组件目录清楚分层 · 本地启动、生产发布和配置边界各归其位

技术结论以本仓库所链接代码和配置为准。Cloudflare 产品概念可另查

Workers [https://developers.cloudflare.com/workers/]、D1 [https://developers.cloudflare.com/d1/] 与 R2 [https://developers.cloudflare.com/r2/] 官方文档；外部项目的业务或技术选择没有移植为本项目事实。